

OpenStack Production Troubleshooting Guide

Practical commands and good practices for diagnosing VM problems in production

control plane · read-only

illustrative

```
$ openstack server show app-17 -c status -c OS-EXT-STS:task_state -c OS-EXT-SRV-ATTR:host
status                BUILD
OS-EXT-STS:task_state  networking
OS-EXT-SRV-ATTR:host   compute-042
# The request reached a host and is waiting on the network side.
# Next question: is the port bound on compute-042, and is its agent alive?
```

HOW TO USE IT

- ☐ Start with section 1. Nothing in it changes production.
- ☐ Pick the symptom, follow its path in order, stop where the evidence changes.
- ☐ Every command answers a question. If you cannot name the question, do not run it.
- ☐ Commands are read-only unless tagged **CHANGES STATE**.

CONTENTS

01	Before touching anything	2
02	VM troubleshooting: first commands	3
03	VM is ACTIVE but unreachable	4
04	VM stuck in BUILD / ERROR	5
05	VM cannot boot: compute problems	6
06	VM networking	7
07	Storage problems	8
08	Logs: where to look	9
09-10	Golden rules · Incident mindset	10
—	The Production Question	11

BASED ON THE PRODUCTION PRINCIPLES DEVELOPED IN

OpenStack, the Day After Tomorrow
Operating Mission-Critical OpenStack Platforms

Soufian Zaouam
OpenStack Production Engineer / Architect

01 Before touching anything

The first ten minutes decide whether you investigate a platform or damage it. Nothing on this page changes production.

Understand the business impact

What is broken, for whom, and how badly?

Which capability: create, reach, attach, migrate?
Which workloads and owners? Is it getting worse?

Identify the affected scope

One VM, one project, one host, or everyone?

Compare with a known-good VM. List what else runs on the same host: `openstack server list --all-projects --host <compute>`

Isolated or systemic?

Does a fresh test VM fail the same way?

One failing VM is a case. Many failing VMs is a platform problem: check `compute service list` and `network agent list` before the VM.

No change before understanding

What would this action destroy?

A restart clears queues, caches and in-memory state, and with them the evidence you still need.

Check recent changes

What changed in the last 24–72 hours?

Deployments, configuration, host maintenance, certificates, firewall and fabric changes, flavor, image and quota edits. “What changed?” resolves more incidents than any metric.

Preserve evidence

What will I need to explain this tomorrow?

Save command output with UTC timestamps, request IDs and UUIDs. Copy logs before a restart or rotation.

Define the rollback before the change

If this makes it worse, how do I get back?

Write down the stop condition and the way back before running anything. No way back means the decision belongs to someone accountable.

Stop sign. If the only way to get the evidence you need is to change production, you have reached a decision point, not a diagnostic step.

Scope tells you where to look first

SCOPE OBSERVED	SUSPECT FIRST
One VM	The guest, its port, its volume, its flavor and image properties
One project	Quotas, security groups, project network and router, RBAC / policy
One compute host	nova-compute, OVS agent or ovn-controller, libvirt, host network and storage paths
One network, rack or AZ	Network node or gateway chassis, fabric, storage backend segment
Every new request fails	Control plane: APIs, RabbitMQ, MariaDB, Keystone, Placement
Running VMs affected too	Data plane and shared infrastructure: fabric, storage backend, power, hypervisors

02 VM troubleshooting: first commands

Eleven commands cover most first looks. Each one answers a specific question. All run from the control plane and are read-only; several fields require admin credentials.

COMMAND	CONTROL PLANE	QUESTION IT ANSWERS	SIGNAL TO LOOK FOR
<code>openstack server show <vm></code>		Where is this VM, in what state, and what failed?	vm_state / task_state / power_state, fault (ERROR only), OS-EXT-SRV-ATTR:host, flavor, image, addresses
<code>openstack server list --all-projects --project <project></code> or <code>--host <compute></code>		Is it one VM, or a pattern?	Several VMs in BUILD or ERROR on the same host, flavor or image
<code>openstack server event list <vm></code> then <code>server event show <vm> <request-id></code>		What was done to this VM, when, and with what result?	Last action and its request ID , your key into the logs; failed events and their traceback (admin)
<code>openstack console log show <vm></code>		Did the guest boot, and how far did it get?	Kernel panic, fsck or emergency shell, cloud-init with no DHCP lease or no metadata. An empty log is a signal too.
<code>openstack hypervisor list</code>		Are the hypervisors up and enabled?	State down, status disabled, a host missing from the list
<code>openstack hypervisor show <compute></code>		What does Nova know about this host?	State, status, host IP. Since API microversion 2.88, usage figures and VM counts are no longer returned: ask Placement.
<code>openstack compute service list</code>		Is Nova alive, and are the computes reporting?	nova-compute down (stale heartbeat), disabled with a reason, conductor or scheduler down
<code>openstack volume show <volume></code>		Is the volume where Nova thinks it is?	status (in-use, attaching, error_attaching...), attachments, os-vol-host-attr:host
<code>openstack port show <port></code>		Is the VM's port bound, and up?	status ACTIVE / DOWN, binding_vif_type = binding_failed, binding_host_id ≠ VM host, security groups
<code>openstack network show <network></code>		Is the network what the VM expects?	Network type and segment, mtu, subnets, port_security_enabled, admin state
<code>openstack server show --diagnostics <vm></code>		What does the hypervisor report right now?	CPU time, memory, NIC and disk counters (admin). Counters that never move: idle, hung, or disconnected.

Two habits. Use `-c <field>` to keep only what answers your question, and `-f json` when you save evidence. Diagnostics are an option of `server show`; the client has no separate `server diagnostics` command.

Find the VM's other identities first. `openstack port list --server <vm>` gives the port UUIDs; `openstack server volume list <vm>` gives the volumes; `-c OS-EXT-SRV-ATTR:instance_name` gives the libvirt domain name.

03 VM is ACTIVE but unreachable

ACTIVE means Nova believes the VM is running. It says nothing about the network path or the guest. Walk the path in order, and stop where the evidence changes.

- 01 VM state** *Is it really running where Nova says?* CONTROL PLANE NODE

server show: power_state Running, host, instance_name · on the host: virsh domstate <instance_name>
- 02 Compute host** *Are the host and its network agent healthy?* CONTROL PLANE

compute service list --host <compute> · network agent list --host <compute>: OVS agent or OVN controller alive
- 03 Neutron port** *Is the port bound on that host, and ACTIVE?* CONTROL PLANE

port list --server <vm> · port show: status, binding_vif_type ≠ binding_failed, binding_host_id = VM host
- 04 Network / subnet** *Is the addressing plan what the guest expects?* CONTROL PLANE

subnet show: gateway, host routes, DNS, DHCP enabled · network show -c mtu: a wrong MTU lets ping through and breaks SSH or TLS
- 05 Security groups** *Is this traffic allowed?* CONTROL PLANE

port show -c security_group_ids -c port_security_enabled · security group rule list <sg>: direction, protocol, port, remote prefix
- 06 Floating IP** *Is the floating IP attached and routable?* CONTROL PLANE

floating ip show: port_id, fixed_ip_address, status · the router has an external gateway
- 07 DHCP & metadata** *Did the guest get its address and its configuration?* CONTROL PLANE NODE

Console log: cloud-init lease and metadata lines · OVS agent: DHCP agent alive, qdhcp-<net> namespace · OVN: DHCP answered by ovn-controller on the compute
- 08 Router** *Is the router up, and hosted where it should be?* CONTROL PLANE

router show: external_gateway_info, interfaces · OVS agent: network agent list --router <router> --long (HA state) · OVN: gateway chassis in ovn-sbctl show
- 09 OVS / OVN dataplane** *Do packets leave the tap and match the right flows?* NODE

tcpdump -eni tap<port-id[:11]> · ovs-ofctl dump-flows br-int (counters moving?) · ovs-appctl ofproto/trace or ovn-trace
- 10 Guest OS** *Is the guest itself refusing?* CONTROL PLANE

Console log, console url show <vm>: interface up, address, routes, guest firewall, sshd listening
- 11 Test from several points** *Where exactly does reachability stop?* NODE

Same-network VM → qrouter- / qdhcp- namespace (OVS agent) → external client. The last point that works names the layer.

Security groups are not the first suspect by default. A port that is DOWN or unbound, or an agent that is dead, produces the same symptom. Check steps 01–03 before you read rules.

04

VM stuck in BUILD / ERROR

Do not start with “what failed?”. Start with “where did the request stop?”. `task_state` and `host` answer it before any log does.

TASK_STATE	HOST SET?	THE REQUEST STOPPED IN	LOOK AT
scheduling	no	Scheduler, Placement, conductor, RPC	nova-scheduler and nova-conductor logs · allocation candidates · RabbitMQ health
networking	yes	Port creation and binding	neutron-server · agent or ovn-controller on that host
block_device_mapping	yes	Volume creation or attachment	Volume status · cinder-scheduler, cinder-volume, backend
spawning	yes	Image download, libvirt/QEMU start, wait for network-vif-plugged	nova-compute on that host · libvirt · QEMU instance log
none, status ERROR	either	The build was aborted	fault in server show · server event show

control plane · read-only

```
# 1. where did it stop?
openstack server show <vm> -c status \
  -c OS-EXT-STS:task_state \
  -c OS-EXT-SRV-ATTR:host -c fault
openstack server event list <vm>
openstack server event show <vm> <request-id>
# 2. could any host take it?
#   (osc-placement plugin)
openstack allocation candidate list \
  --resource VCPU=4 --resource MEMORY_MB=8192 \
  --resource DISK_GB=40
openstack resource provider allocation show \
  <vm-uuid>
nova-manage cell_v2 list_hosts
# 3. what did the request ask for?
openstack flavor show <flavor> -c properties
openstack image show <image> -c status \
  -c disk_format -c min_disk -c properties
openstack port list --server <vm>
openstack quota show <project>
# 4. follow the request ID in the logs
#   nova-api → conductor → scheduler → compute
```

Faults you will meet, and what they mean

- No valid host was found.**
No host passed Placement and the filters. Check candidates for this flavor, disabled services, AZ and aggregate filters, NUMA / PCI requests, hosts not mapped to a cell.
- Host '...' is not mapped to any cell**
Capacity exists but nothing lands on it. `nova-manage cell_v2 discover_hosts --verbose` **CHANGES STATE**
- Exceeded maximum number of retries.**
The build failed on several hosts. The cause is on the hosts: read nova-compute on each, not the scheduler.
- Failed to allocate the network(s), not rescheduling.**
network-vif-plugged never arrived before `vif_plugging_timeout`: port binding, agent or ovn-controller, RabbitMQ.
- Volume ... did not finish being created**
Boot from volume waited for Cinder and gave up: backend slow or failing, large image-to-volume copy.
- Quota exceeded for ...**
Usually rejected by the API before any VM exists. `quota show, limits show --absolute --project`.

Resetting state is not a fix. `openstack server set --state` **CHANGES STATE** rewrites the record, not the reality. Use it only once you know what the hypervisor, Neutron and Cinder actually hold. Check recent flavor, image, aggregate and host changes first.

05 VM cannot boot: compute problems

Once the scheduler has picked a host, the question changes from “is there capacity?” to “can this host honour what the flavor asks for?”

● ● ● first, map the VM to its host and its libvirt domain

```
cp$ openstack server show <vm> -c OS-EXT-SRV-ATTR:host -c OS-EXT-SRV-ATTR:instance_name
node$ virsh domstate instance-0000abcd # the name Nova gave the domain, not the VM name
```

Compute service CONTROL PLANE

Can this host receive work at all?

```
compute service list \
  --service nova-compute --long
```

Up and enabled, disabled reason. A host that is up but disabled receives nothing.

Huge pages NODE

Are there enough free pages on the right NUMA node?

```
grep -i huge /proc/meminfo
/sys/devices/system/node/node*/
  hugepages/*/free_hugepages
```

Pages are counted per node: a large free total can still fail.

Resources: Placement is the source of truth

CONTROL PLANE

What does the scheduler believe is free?

```
resource provider list --name <compute>
resource provider inventory list <rp>:total,
reserved, allocation_ratio
resource provider usage show <rp>
nova-manage placement audit --verbose:allocations
with no instance. Orphan allocations make a host look full.
```

PCI / SR-IOV CONTROL PLANE NODE

Is the device exposed, free, and matched?

Port binding: vnic_type=direct; [pci] device_spec (passthrough_whitelist before Zed) and alias; host: lspci -nn, ip link show <pf> for VFs. These VMs often cannot be live-migrated.

CPU, RAM, NUMA NODE

Does the guest topology fit one host NUMA node?

Flavor: hw:numa_nodes, hw:mem_page_size. Host: numactl --hardware, lscpu. CPU and memory of a guest NUMA node must come from the same host node.

libvirt NODE

Did libvirt define and start the domain?

```
virsh list --all, virsh dominfo, virsh dumpxml
(devices, CPU tune, disks)
journalctl -u libvirtd (or virtqemud with modular daemons)
```

CPU pinning NODE

Are dedicated CPUs available, and really dedicated?

hw:cpu_policy=dedicated consumes PCPU from [compute] cpu_dedicated_set. Check virsh vcpupin and virsh emulatorpin. Nova pins the vCPUs; isolating the host from those cores is your job.

QEMU and instance logs NODE

Why did the process refuse to start, or die?

/var/log/libvirt/qemu/<instance_name>.log: missing device, CPU model, permissions, SELinux / AppArmor denials. Then nova-compute on that host, filtered by request ID.

Containerised deployments. Run virsh inside the libvirt container and read logs from the deployment tool's log directory; the questions do not change.

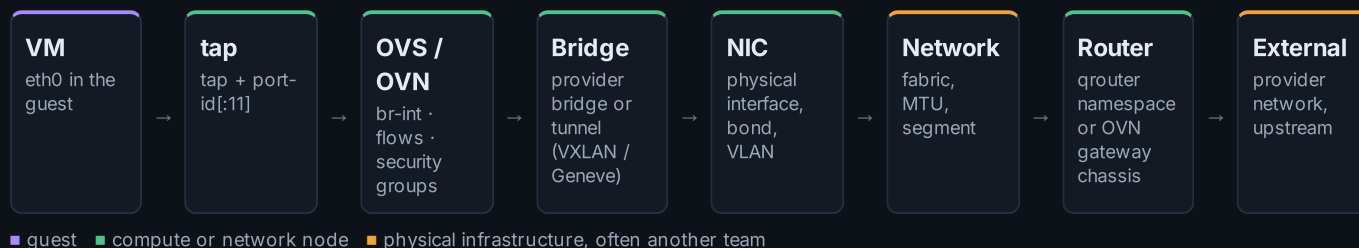
AN ORDER THAT SAVES TIME

Service enabled → Placement capacity → Flavor fits host → libvirt domain → QEMU process

Each step is cheaper than the next, and each one rules out a whole layer. Jumping to the QEMU log first often explains the last error, not the first one.

06 VM networking

Follow one packet. Every hop has a place to look, and the first hop where it disappears is your layer.



CONTROL PLANE

what OpenStack believes

```

openstack port show <port>
# bound? on which host? ACTIVE? which SGs?
openstack network show <network>
# type, segment, MTU
openstack subnet show <subnet>
# gateway, DHCP, host routes
openstack router show <router>
# external gateway, interfaces
openstack floating ip show <floating-ip>
# attached to which port?
openstack network agent list --host <node>
# agents alive on that node?

# OVN: on a node with NB/SB DB access
ovn-nbctl show
# logical switches, routers, ports
ovn-sbctl show
# chassis and where ports are bound
ovn-trace <datapath> '<microflow>'
# what the logical pipeline does
  
```

COMPUTE / NETWORK NODE

what the host does

```

ip link | grep tap<port-id[:11]>
# tap exists and is UP?
ip addr ; ip route
# underlay addressing and routes
ovs-vsctl show
# bridges, ports, tunnels
ovs-ofctl dump-flows <bridge>
# run twice: are n_packets moving?
ovs-appctl ofproto/trace <bridge> <flow>
# which rule drops the packet?
tcpdump -eni tap<port-id[:11]>
# does traffic leave the VM?
tcpdump -eni <nic> <filter>
# does it reach the wire?

# OVS agent (ML2/OVS)
ip netns
# qrouter-<id>, qdhcp-<id>
ip netns exec qdhcp-<net-id> ping <vm-ip>
# OVN
ovn-appctl -t ovn-controller connection-status
  
```

MTU. Overlays subtract encapsulation overhead. Ping works, SSH hangs after the banner, TLS stalls: suspect MTU before security groups.

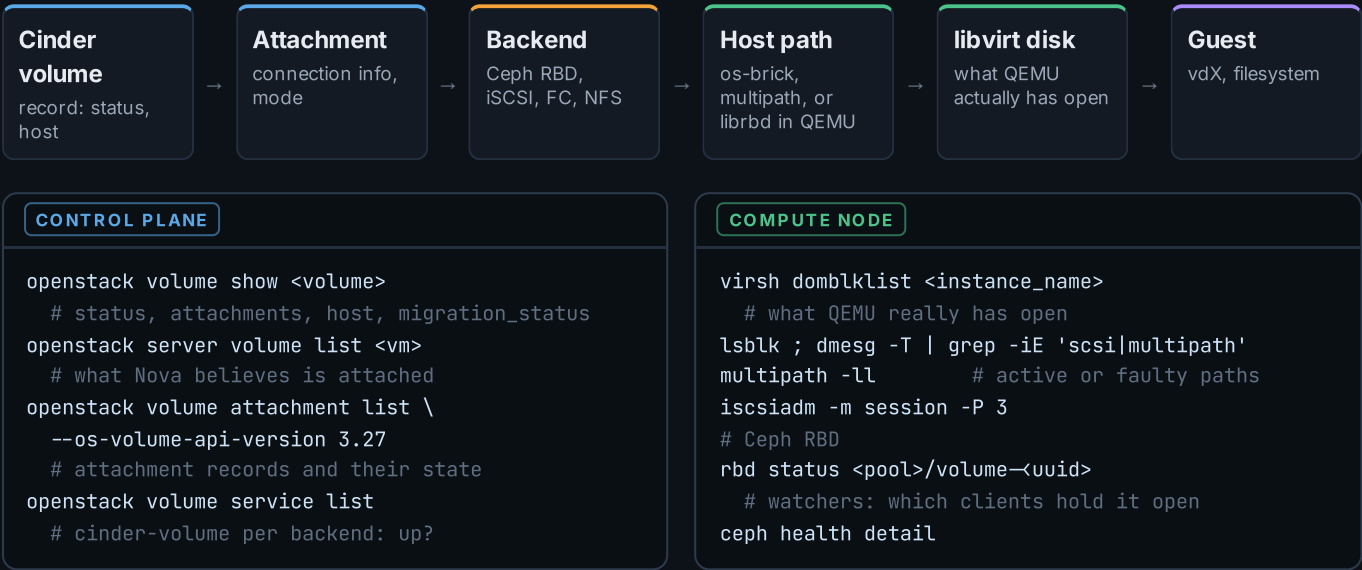
Counters, not rules. A flow whose n_packets never moves is not on the path, however plausible it looks.

Reachability matrix: where does it stop?

WORKS FROM	FAILS FROM	SUSPECT
Nowhere, not even the same network	—	Guest (address, interface, firewall), tap / port binding, security groups on the port
Same network only	Router, external	Gateway in the guest, router interface, router hosting (L3 agent / gateway chassis)
Router namespace or gateway	External	Floating IP / SNAT, external network, upstream routing, fabric
Small packets (ping)	Large packets (SSH, TLS)	MTU along the overlay or the physical path
Other hosts' VMs	VMs on one host	That host: agent / ovn-controller, tunnel endpoint, NIC, bond

07 Storage problems

A volume exists in three places: the Cinder record, the Nova block device mapping, and the hypervisor. Most storage incidents are these three disagreeing.



08

Logs: where to look

Logs live under `/var/log/<service>/`, in `journald`, or in the deployment tool's log directory. Knowing which component to open saves more time than any `grep`.

COMPONENT	PROCESSES	WHAT TO INVESTIGATE
Nova	api · scheduler · conductor · compute	Request accepted or rejected; NoValidHost and filter results; build orchestration and RPC timeouts; spawn, vif plugging, volume attach
Neutron	server · agents · OVN (northd, controller)	Port binding failures; agent heartbeats and flow programming; OVN database connectivity and translation
Cinder	api · scheduler · volume	Capacity and filter decisions; backend driver errors; attach and detach
Keystone	keystone	Authentication: 401 / 403, token validation, Fernet keys out of sync between nodes
Glance	api	Image upload and download, store backend errors, format and checksum
Placement	api	Allocation claims, 409 conflicts, inventory updates from computes
RabbitMQ	server	Messaging: partitions, memory and disk alarms, connection churn, queues without consumers
MariaDB	server (Galera)	Database: cluster state changes, flow control, deadlocks, connection limits
libvirt	libvirtd / virtqemud	VM lifecycle: domain define and start, device attach, migration
QEMU	per-instance log	VM execution: start failures, crashes, device and permission errors
OVS / OVN	ovs-vswitchd · ovsdb-server · ovn-controller	Networking: port and flow errors, OVSDb connectivity, chassis registration

Correlate, or it is not evidence

Timestamps

In UTC, from clocks you trust. Two minutes of skew reorders the story.

Request IDs

req-`<uuid>` on every line; `global_request_id` carries the caller's ID into Neutron, Cinder and Glance.

VM UUID

Nova, Placement consumer, `<uuid>` in `virsh dumpxml`.

Port UUID

neutron-server, agents, the tap name.

Volume UUID

Cinder, `os-brick` in `nova-compute`, RBD image `volume-<uuid>`.

Recent changes

Deployment runs, configuration commits, maintenance tickets.

CONTROLLER

one request, several services

```
# the request ID comes from
# openstack server event list <vm>
grep -rh "req-<uuid>" \
  /var/log/nova/ /var/log/neutron/ \
  /var/log/cinder/ | sort

# then the same ID on the compute
# host that ran the build
```

Baseline first.

An ERROR line that appears every day is not your incident. Know what normal noise looks like before you read an outage into it.

09 Production golden rules

01 Understand before changing.

A change made to test a hypothesis is still a production change.

03 Protect what is working.

Running workloads come first. Every action is judged by what it could break.

05 A command is not a diagnosis.

Output is evidence. The diagnosis is the explanation that fits all of it.

07 Correlate symptoms across layers.

A Nova symptom often starts in Neutron, the message bus or the database.

09 Automate repetitive read-only diagnostics first.

Let scripts collect evidence. Keep the judgement human.

02 Stability comes before new features.

Pausing a capability is acceptable. Losing a platform state you can trust is not.

04 Never change production without a rollback strategy.

No way back means the decision belongs to someone accountable.

06 Logs without context are not evidence.

Time, request ID and scope turn a log line into a fact.

08 Document production decisions.

What was known, what was chosen, what was rejected, and why.

10 Do not confuse recovery with root-cause analysis.

Service restored is not problem understood.

10 Incident mindset

Five activities, in this order. They need different speeds, and often different people.

1

Restore service

GOAL
Reduce business impact

DONE WHEN
The capability works again, even degraded

→

2

Stabilize

GOAL
Stop the situation getting worse

DONE WHEN
State has stopped changing; guardrails are in place

→

3

Understand root cause

GOAL
Explain every symptom with evidence

DONE WHEN
The explanation survives a review

→

4

Fix

GOAL
Remove the cause, not the symptom

DONE WHEN
Verified under the conditions that caused it

→

5

Prevent recurrence

GOAL
Change detection, runbooks, design

DONE WHEN
The change exists and has an owner

Recovery and root-cause analysis are different activities. Recovery accepts uncertainty to reduce impact now. Root-cause analysis removes uncertainty, and cannot be rushed without being wrong.

A restart is also a deletion. Before restoring, capture what the restart will erase: queue state, process state, logs about to rotate, the exact error.

The Production Question

"What do I know, what do I not know, and what evidence do I need before touching production?"

WHAT DO I KNOW?

Facts I observed myself, with a timestamp and a source.

Not what the dashboard implies.
Not what worked last time.

WHAT DO I NOT KNOW?

Hypotheses and assumptions, written down as such.

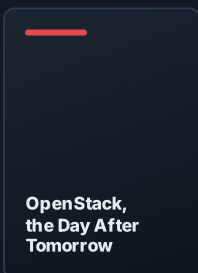
Uncertainty is a risk signal, not a weakness to hide.

WHAT EVIDENCE DO I NEED?

The smallest safe observation that would change the decision.

If it can only be obtained by changing production, it is a decision.

Commands answer questions. Experience is knowing which question to ask next, and when the honest answer is: not yet.



BASED ON THE PRODUCTION PRINCIPLES DEVELOPED IN

OpenStack, the Day After Tomorrow

Operating Mission-Critical OpenStack Platforms

This guide is the field companion: what to check, and in which order. The book is the deeper reference: why control over a platform erodes, how to assess it, and how to decide when the answer is not obvious.

github.com/soufian-zaouam/openstack-the-day-after-tomorrow

Soufian Zaouam

OpenStack Production Engineer / Architect

v1.0 · September 2026. Commands are read-only unless tagged "changes state". Options and output fields vary with release, client version and API microversion: verify against your deployment. Independent work, not affiliated with the OpenInfra Foundation. OpenStack is a trademark of the Open Infrastructure Foundation.